

Robot state: estimation and admissible control

M11 • 08 Aug 2027 - 12 Sep 2027 • 75 hours

This month's deliverable

You will connect three engineering questions: what is happening inside the mechanism, what can be reconstructed from sensors, and which command is admissible to send. The deliverable compares PID, LQR, estimation with KF and EKF, and a simple QP on a model with known parameters, using identical disturbances and testing optimiser failure. For a leg, this prepares for contact-force allocation; for a finger, it prepares for control using an estimated state with bounded actuation.

Begin with the axis from M10. State $x = [q, v]$ contains angle q in rad and angular velocity v in rad/s; input u is torque in N·m; the encoder measures angle. In this model, velocity can be reconstructed from a sequence of measurements even without a separate velocity sensor. In a small separate experiment, estimate tilt from a gyroscope and accelerometer. Then solve a planar force-allocation problem with two supports. These three parts share rules for time, units and checks, but each needs its own mathematical model.

Prerequisites

Review matrix multiplication, derivatives, eigenvalues and the meaning of variance. Bring the baseline PID, joint model, command limiter and fixed test scenarios from M10. Distinguish measurements, simulation ground truth and estimates. Validate estimates against a reference: the known state in simulation or readings from an independent reference system. The output of an IMU-processing library is not, by itself, true orientation.

Limits on complexity

This month does not require a complete MPC, a general-purpose 3D-EKF or whole-robot balancing. Required topics are state space, linearisation, controllability and observability, the linear Kalman filter, a small tilt EKF, LQR and a convex quadratic programme (QP) with explicit constraints. Mathematics should explain the experiment rather than block it for weeks. You will check each step from model to solution and see the limitations of each approximation.

The final project contains code, equations, units, covariances, chosen objective weights, raw logs and automated checks. All numerical thresholds in example models are experiment parameters. For a physical actuator, limits come from its specifications, load and the previously validated rig. "The optimiser found a solution" implies neither physical feasibility nor a correct model.

Organising the 75 hours

Four work blocks

Block A, hours 1-20: state space and observability. Block B, 21-40: KF, EKF and estimation quality. Block C, 41-60: LQR and a comparison with PID under identical conditions. Block D, 61-75: force QP, faults and integration. Each of the first three blocks contains 10 hours of theory and analysis and 10 hours of practice; the fourth contains 5 hours of analysis and 10 hours of practice. Total: $35 + 40 = 75$ hours. The work steps on pages 9-10 expand these same hours and require no additional time allocation.

Divide each ten-hour theory block into five two-hour sessions: 20 minutes reviewing the previous model, 45 minutes reading the relevant section, 40 minutes deriving a small example or examining a log, and 15 minutes recording the conclusion. Also divide the ten-hour Sunday practical session into five two-hour periods with breaks between them. Breaks are excluded from net study hours. Allocate the final five hours of analysis as $2 + 2 + 1$.

Start with the question, then choose the tool

The main environment is Python, NumPy, SciPy and python-control; use FilterPy for cross-checking and CVXPY and OSQP for a small convex problem. Begin each session with a testable question: how does the input affect the output, which state cannot be measured, and which data reveal its changes? Before running, write down the expected sign, scale or sequence of events. Compare the prediction with the result and examine at least one discrepancy. A plot supports a conclusion only when accompanied by an explanation of the observed relationship.

Maintain one repository with model, config, experiments, tests and reports directories. Every experiment needs a unique identifier, code version, configuration, parameter source, initial state, units and raw data. The analysis script reads a saved experiment and builds the report without manually correcting measurements. The final state-estimation-and-control benchmark project combines comparisons of estimation and control methods; individual exercises gradually form its test suite.

Time accounting and reducing scope

Calendar boundaries may coincide with an adjacent module. Keep rest days, holiday intervals and integration checks as specified in the overall calendar; place numbered steps in available time without counting hours twice. Do not add hours to this module. Assess equipment and library readiness at the outset rather than spending the practical session on an unplanned platform change.

If you fall behind, reduce the number of LQR weight sets, support geometries and validation runs with different random-number seeds after the minimum checks. Retain the axis model with a two-dimensional state, an unobservable case, KF with a missing measurement, tilt EKF with an outlier, an equal-conditions PID/LQR comparison and a two-support QP with infeasible and timeout statuses. Full 3D orientation and MPC are unnecessary. The result must connect modelling, estimation and constrained control; all declared results can be obtained in simulation with known parameters.

State space and the operating point

Study the state and its evolution

Write $\dot{x} = Ax + Bu$, $y = Cx + Du$. For $J\ddot{q} + b\dot{q} = u$, the matrices are $A = \begin{bmatrix} 0 & 1 \\ 0 & -b/J \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ 1/J \end{bmatrix}$, $C = \begin{bmatrix} 1 & 0 \end{bmatrix}$, $D = \begin{bmatrix} 0 \end{bmatrix}$. Dimensions: A 2×2 , B 2×1 , C 1×2 , D 1×1 . Rows have different physical units; matching matrix shapes do not make q and v interchangeable. In discrete time, use $x[k+1] = A_d \cdot x[k] + B_d \cdot u[k]$, obtaining A_d and B_d for the declared T_s with a held command.

For a nonlinear model $\dot{x} = f(x,u)$, choose an equilibrium $f(x_0, u_0) = 0$ and work with deviations δx , δu . Evaluate Jacobians $A = \partial f / \partial x$ and $B = \partial f / \partial u$ at that point. Check the meaning of gravity compensation in u_0 for a loaded axis: do not linearise around an accelerating state and then treat it as stationary. Small-deviation equations apply only within a limited neighbourhood.

Task 1. A one-step transition

Inputs: the M10 model, two initial states and three constant torques within the model's admissible range. 1) Derive the matrices. 2) Obtain A_d and B_d . 3) Compare one step with numerical integration on a fine grid. 4) Repeat for zero torque and non-zero velocity. Deliverable: `state_transition_tests` and an error table. Check: $q=v=u=0$ remains the zero state; for positive torque, the first velocity change has the correct sign. Error to test: omitting T_s from an update creates a scaling error that the test must detect.

Task 2. Observability and controllability

Construct the controllability matrix $[B, AB]$ and observability matrix $[C; CA]$, and calculate rank and singular values. For comparison, replace the angle sensor with a velocity-only sensor $C = [0, 1]$. For a free axis without positional stiffness, absolute angle is then unobservable. Specify two different initial angles with the same velocity and show identical measurements. Deliverable: a short demonstration using two trajectories, as well as `rank()`. Check sensitivity of numerical rank to unit scaling; poor conditioning can hinder estimation even when rank is formally full.

Then linearise a model with gravity near a chosen q_0 and compare it with the nonlinear model for small and large deviations. State the region in which the linear prediction error is acceptable under your predefined criterion.

KF and EKF: estimation with testable uncertainty

Study filter prediction and update

Prediction calculates the next estimate and covariance from the model; the update refines them using a new measurement. Q describes process-model uncertainty, R measurement uncertainty, and P estimate uncertainty. These are covariances, not “confidence coefficients between zero and one”. Their dimensions correspond to products of component units. If T_s changes, reconsider discrete Q using the chosen noise model. Scalar angle variance has units rad^2 ; angle-velocity covariance has units rad^2/s .

The measurement residual, or innovation, $v = z - h(\hat{x}^-)$ is the difference between a measurement and its prediction; S is the predicted covariance of that difference. A smooth estimate can be wrong even when its claimed uncertainty is small. Check error against simulation ground truth, changes in P and innovation statistics. For NIS, calculate $v^T S^{-1} v$ by solving a linear system without explicitly inverting the matrix. Comparison with a χ^2 distribution is meaningful under correct Gaussian assumptions and the appropriate measurement dimension. This is a diagnostic check with stated conditions of applicability.

Task 3. Angle and velocity from a noisy encoder

Generate joint motion with a fixed seed, encoder noise and a small model error. Implement prediction and update for $x=[q,v]$; compare with a library using identical data. Compare the velocity estimate with a simple finite difference. Save true and estimated q and v , the diagonals of P , the innovation and RMSE. Check: noisy data are not written to truth; when a measurement is missing, perform prediction only and let the corresponding uncertainty grow. Then set R artificially 100 times smaller than the actual variance and explain the consequences using innovations and estimation error.

Task 4. An EKF for one tilt axis

For one axis, choose state $[\theta, b_g]$: tilt in rad and gyroscope bias in rad/s . Prediction: $\theta[k+1] = \theta[k] + T_s \cdot (\omega_m - b_g)$, $b_g[k+1] = b_g[k]$. For a consistent axis, you may use accelerometer model $h=[g \cdot \sin\theta, g \cdot \cos\theta]$ in m/s^2 ; explicit signs must match the coordinate-frame convention and rest test. Derive $H=[g \cdot \cos\theta, 0; -g \cdot \sin\theta, 0]$. Check the Jacobian with finite differences. Introduce an outlier and translational acceleration; innovation gating must mark rejected measurements and preserve the prediction. Deliverable: an experiment with gyroscope drift, accelerometer correction and temporary rejection of updates; yaw cannot be determined from gravity direction alone.

LQR and LQG: the cost of error and control effort

Study the objective function

For the discrete axis, define the sum $x^T Q x + u^T R u$ and find feedback $u = -Kx$. Q is positive semidefinite and R is positive definite; do not confuse this Q with the filter's process covariance. Name them Q_cost and Q_noise in code. To compare rad, rad/s and N·m meaningfully, normalise components by selected characteristic scales q_ref , v_ref and u_ref . The weights then express the relative cost of error and actuation, rather than depending accidentally on a conversion to degrees.

LQR does not impose saturation itself. On a real actuator, a limiter changes the closed-loop dynamics, so test it separately. Tracking a non-zero reference requires deviations from an equilibrium and the corresponding u_0 ; simply inserting absolute x into $-Kx$ drives it towards zero. LQG combines a linear Kalman filter with LQR. The separation principle applies under specific linear conditions; it provides no automatic stability guarantee for delays, saturation, nonlinearities or EKF.

Task 5. LQR on an axis with true state

Use A_d and B_d from task 1. 1) Specify two sets of normalised weights: prioritising accuracy and prioritising small torque. 2) Calculate K and the poles of $A_d - B_d K$. 3) Check stabilisation at zero and a small step change of equilibrium. 4) Add the same limit on u . Deliverable: two responses and a table of RMSE, peak command and the integral $u^2 dt$ in $N^2 \cdot m^2 \cdot s$. Check: predicted poles are inside the unit circle for the linear case; under saturation, support the conclusion with a separate experiment. Failure case: weights designed for degrees are applied to radians without conversion.

Task 6. Comparing PID and LQR under identical conditions

Take the PID with frozen settings from M10. First compare it with LQR under equal access to true state or measurements; state explicitly which controller receives velocity. Then connect KF and repeat the noisy cases. Use the same plant, limits, T_s , initial conditions, disturbance and test seeds. Deliverable: a table of accuracy, control effort, computation time and saturation fraction. Check: tuning uses a separate scenario set; the final set is not used to fit weights. If LQR wins through access to true velocity, describe this as an information advantage rather than an advantage of the algorithm itself.

Separately compare an inaccurate model and estimation delay. Make the conclusion specific: which controller is better for the chosen axis and conditions, where behaviour differs and which added complexity is justified. LQR superiority is not a required outcome of the exercise.

QP: allocating forces between two supports

Variables, geometry and dimensions

Model: two point supports in the x-z plane, with coordinates relative to the body centre $r_1=(-a,-h)$, $r_2=(a,-h)$, where a,h are in metres. The unknown vector is $f=[f_{x1},f_{z1},f_{x2},f_{z2}]^T$; all four components are in newtons. Contact pushes upward: $f_{zi} \geq 0$. The prescribed force-and-moment vector, or wrench, $w=[F_x,F_z,M_y]$ has component units N, N and N·m. From $r \times f$, obtain $M_y=-h \cdot f_{x1}+a \cdot f_{z1}-h \cdot f_{x2}-a \cdot f_{z2}$. Thus A is 3×4 with third row $[-h,a,-h,-a]$; the first rows sum f_x and f_z .

To avoid directly adding squared N and N·m values, define $D=\text{diag}(1/F_{ref},1/F_{ref},1/M_{ref})$. Minimise $\|D(Af-w)\|^2 + \lambda \|f/F_{ref}\|^2$ with $\lambda > 0$. Constraints: $0 \leq f_{zi} \leq F_{max_i}$ and $-\mu f_{zi} \leq f_{xi} \leq \mu f_{zi}$; μ is dimensionless. This is a planar friction cone with linear boundaries and a convex QP. Label $a,h,F_{ref},M_{ref},F_{max}$ and μ as illustrative values or measurement-derived values. Contact forces do not yet convert automatically into real actuator commands; that will later require Jacobians and model validation.

Task 7. A feasible allocation

Take symmetric geometry and request $F_x=0$, $F_z=mg$, $M_y=0$. 1) Write A and the bounds. 2) Solve the QP. 3) Check $f_{z1}=f_{z2}$ for symmetric limits. 4) Add a small M_y and explain the load transfer. 5) Increase F_x until the friction boundary is reached. Deliverable: a computational notebook, forces at each support, a normalised residual and active constraints. Check: independently recalculate Af , every bound and the moment sign. Do not trust solver status alone: check residuals in physical units with an explicit tolerance.

Task 8. An unreachable request and solver failure

The problem with the soft objective above usually has a feasible solution $f=0$, so separately add the equality $Af=w$ and require $F_z > F_{max1}+F_{max2}$ to create a deliberately infeasible request. Then check premature termination at the iteration limit and an artificially late result. Record status, residual, iteration count and time; compare cold and warm starts on the same request sequence. Deliverable: p50, p95, p99, maximum and the fraction of unusable solutions. The fallback mode uses only a previously validated action for the current constraints; a previous solution cannot be repeated unconditionally after losing contact.

A common experiment and component compatibility

Three connected experiments

Part A - axis: model $\rightarrow$ sensor $\rightarrow$ KF $\rightarrow$ PID or LQR $\rightarrow$ limiter $\rightarrow$ model. Part B - tilt: gyroscope and accelerometer $\rightarrow$ EKF $\rightarrow$ log of estimates, errors and rejected updates. Part C - force allocation: requested wrench $\rightarrow$ QP $\rightarrow$ independent constraint check $\rightarrow$ admissible output or fallback mode. Align units, timing and scenario format. QP forces cannot be inserted into the joint model without a physically justified transformation; a full model of a balancing robot is not required this month.

The axis scenario contains J , b , T_s , noise, initial state, u limits, seed and reference trajectory. The EKF scenario contains the θ trajectory, true gyroscope bias, noise intensities and intervals of additional acceleration. The QP scenario contains support coordinates, μ , force limits, requests and time limits. Label all synthetic parameters as synthetic; measured contains only physically measured data.

Meaningful metrics

For the axis, calculate angle and velocity RMSE, peak command, the integral of squared torque and saturation fraction. For state estimation, calculate RMSE against simulation ground truth, innovation mean and temporal correlation, NIS and the fraction of rejected updates. For QP, calculate the wrench residual in physical and normalised units, maximum bound violation, solution time, iteration count, status and the number of transitions to fallback mode. Laptop timing applies only to that machine and configuration and does not guarantee the same speed on a microcontroller.

Use one set of seeds for parameter selection and a separate held-out set for final validation. Before increasing the run count, try small limiting cases: zero state, zero load, symmetric supports and a missing measurement. Save at least one case you cannot explain and make it an open task with a minimal reproducing example.

Files and reproduction

The README describes mathematical models and applicability limits; config stores units and normalising scales; tests checks transitions, Jacobians, matrix dimensions and constraints. reports contains matching plots for all compared algorithms and a separate fault page. The reproduction command must recalculate aggregates from saved logs; repeating a simulation with pinned versions and seed must give statistically and numerically consistent results. Do not require bitwise equality across different BLAS libraries, operating systems and solver versions without a separate check.

Six tests for incorrect solutions

Estimation and modelling

F1 - unobservable absolute angle. Two states differ only in q ; the sensor sees v on a free axis. Expected: identical measurements; the estimator must not confidently “recover” an unknown offset without additional information. Deliverable: two trajectories and an explanation of which sensor or constraint would change the situation.

F2 - missing encoder samples. Remove several consecutive measurements while retaining the true motion. Expected: prediction continues, updates are skipped and uncertainty changes according to the model. An incorrect variant repeats an old measurement as new and artificially reduces P ; include it as a negative test.

F3 - an IMU outlier and linear acceleration. Introduce one large outlier and a separate sustained acceleration interval. Expected: gating flags the outlier; do not hide the sustained effect by claiming that an accelerometer always measures gravity alone. Compare the number of rejected updates and accumulated drift; separately record resumption of updates.

Control and optimisation

F4 - inaccurate model and LQR saturation. Increase J , add delay and constrain u to the same limit as PID. Expected: measurable changes; the original model’s linear poles are not used as proof of stability for the saturated system. The report shows where the chosen controller’s advantage disappears.

F5 - an infeasible QP with hard equality constraints. Request a vertical force above the sum of the limits with $Af=w$. Expected: an unusable status is handled before publishing a command; the log contains a reason rather than an empty vector. Alongside it, show the soft problem with a non-zero residual to distinguish an optimal approximation from exact fulfilment.

F6 - a late or non-finite solution. Substitute NaN for the output or delay it until the next step. Expected: checks of finiteness, compatibility with current contacts and the computation deadline reject the result. Also recheck the last admissible solution against new constraints; if a support has disappeared, its old forces are no longer permitted.

Analysis and correction

For each test, save inputs, the expected event and the actual event. Check dimensions and normalisation first, then the model, and only then Q , R or solver settings. Do not reduce R merely because an estimate looks smoother, or weaken physical constraints to remove an infeasible status. The corrected version must pass the same original scenario; any criterion change requires an explanation and a new experiment identifier.

Hours 1-40: modelling and estimation

Hours 1-20: state space

Theory 1-10: 1-2: select states, inputs and outputs and list units; 3-4: derive A, B, C and D; 5-6: work through Ad, Bd and the matrix transition; 7-8: controllability and observability; 9-10: operating point and Jacobians. These are five two-hour sessions. End each with a dimensional check and one limiting case for your axis.

Practice 11-20: 11-12: transfer the M10 model; 13-14: complete task 1; 15-16: complete task 2 with two sensors; 17-18: check linearisation through finite differences and trajectories; 19-20: save a validation set of transitions. The ten-hour practical session ends with executable checks, as well as equations in your notes.

Before proceeding, verify that you can explain observable and unobservable states, have correct matrices and have defined the linear model's operating region. Move on to Q and R units after understanding the model.

Hours 21-40: Kalman filter and tilt EKF

Theory 21-30: 21-22: mean and covariance; 23-24: prediction and update; 25-26: innovation and tuning Q_{noise} and R_{noise} ; 27-28: EKF and the measurement Jacobian; 29-30: innovation gating and statistical consistency. Derive one scalar update by hand and trace how changing R changes the correction.

Practice 31-40: 31-32: implement the linear KF in task 3; 33-34: test missing measurements and incorrect R; 35-36: build the tilt model in task 4; 37-38: check the Jacobian, outlier and acceleration; 39-40: plot errors, innovation and P, and write an explanation. Use a library for cross-checking; the central requirement is to explain every input and failure mode.

Before proceeding, check that estimates are compared against an independent reference and that simulation ground truth is stored separately. Explain why a smooth estimate may be wrong and why yaw cannot be determined from gravity direction alone.

Hours 41-75: optimal control and constraints

Hours 41-60: LQR and comparison

Theory 41-50: 41-42: quadratic objective; 43-44: state and command normalisation; 45-46: discrete LQR and feedback; 47-48: tracking relative to equilibrium; 49-50: LQG and the limits of the separation principle. Describe PID and LQR objectives in the same terms so that the comparison addresses the same task.

Practice 51-60: 51-52: complete task 5 with two weight sets; 53-54: add limits and an inaccurate model; 55-56: complete task 6 without noise; 57-58: enable KF and the held-out seeds; 59-60: compile a table of the trade-off between accuracy and control effort. If the result is unexpected, check which information each controller receives before attempting to retune.

Before proceeding, verify that the baseline PID is unchanged, the comparison is reproducible and objective-function dimensions are understood. Tie any conclusion about winning or losing to its conditions, and explicitly mark saturated segments.

Hours 61-75: QP and solver failure

Analysis 61-65: 61-62: two-support geometry and matrix A; 63-64: normalised quadratic objective, friction and infeasibility; 65: solver and fallback requirements, and the final acceptance plan. The last theory block is 2 + 2 + 1 hours; learning general MPC is outside its scope.

Practice 66-75: 66-67: complete task 7; 68-69: complete task 8 with infeasible hard constraints; 70-71: compare warm and cold starts and test timeout and NaN; 72-73: run the six fault tests and compile the common report; 74-75: reproduce the package and prepare interface descriptions for ROS2. If time is limited, reduce geometry count while retaining independent checks of constraints and late results.

Before proceeding, verify that the three package components agree on timing and units, QP does not output unchecked forces and every result has provenance. M12 receives clear data and fault interfaces.

Acceptance: explain and reproduce the solution

What to demonstrate without prompting

Begin with the axis's two-dimensional state and give every matrix dimension. Show an unobservable case, then a successful KF and a poor covariance choice. Explain the EKF Jacobian and demonstrate its comparison with finite differences. Next, show an identical test for PID and LQR and a solved QP with a manual check of total force and moment. Finish with a forced solver failure and the log of transition to fallback mode.

Readiness criteria: state-transition tests pass; every sensor has a measurement model and units; covariance is not treated as an arbitrary confidence percentage; innovations and missing measurements have been investigated; tuning is separate from validation; QP constraints are checked independently; timeout, infeasible and non-finite results are handled. Justify how well innovation statistics agree with the assumed noise model. A histogram's appearance alone is insufficient.

Self-check questions

1) Why can two different states produce the same measurement? 2) How does poor conditioning differ from formal rank deficiency? 3) When is linearisation needed, and how do you check its validity region? 4) Why can a filter not keep updating indefinitely with an old number after a packet is lost? 5) How does Q_{noise} differ from Q_{cost} ? 6) What are the units of R for an accelerometer? 7) Why does increasing the filter's confidence not guarantee a smaller error? 8) Where is LQR action limited? 9) What changes when an angle is converted to degrees without normalisation? 10) Why can a QP with a soft residual penalty remain feasible for an infeasible request? 11) Why do force and moment need different cost scales? 12) When can the last successful solution no longer be used? Relate every answer to a saved negative test.

Handover to the next module

Prepare field-level definitions of `StateEstimate`, `SensorSample`, `ControlRequest` and `ControlStatus`: timestamp, coordinate frame or axis, units, value, covariance or quality status, and scenario identifier. Do not create ROS interfaces before M12, but record which component produces each field and which state is considered stale. Separately list frequency, admissible age and diagnostic requirements.

Numerical experiments do not close unfinished physical checks. As specified by the final project, this month can be completed entirely on models with known parameters; transfer to real equipment remains a separate, justified step. If reducing scope, retain one linear axis, one tilt EKF and a two-support QP. Complex three-dimensional models and rewriting all algorithms in a third language are optional.

Assigned reading: a focused reading route

Books and lecture notes may be in English; keep the plan, definitions and your own explanations in English. Read only the specified sections within the existing 75 hours.

Core reading and one reference route

- 1. Core:** S. Lall, [Stanford EE363: Linear Dynamical Systems, 2026](#). Slides 9 (discrete systems), 11-12 (controllability and observability), 18 (Infinite horizon linear quadratic regulator), 22 (The Kalman filter). For tasks 1-3 and 5, work through only the equations for your axis.
- 2. Core:** R. Labbé, [Kalman and Bayesian Filters in Python](#). Chapters 04 One Dimensional Kalman Filters, 07 Kalman Filter Math, 08 Designing Kalman Filters, 11 Extended Kalman Filters. For tasks 3-4: one manual update step, covariance selection and a Jacobian check.
- 3. Core, selected sections:** S. Boyd, L. Vandenberghe, [Convex Optimization](#). §4.4 Quadratic optimization problems and §5.5 Optimality conditions. For task 7, distinguish the objective from the constraints and establish which properties make the two-support QP convex.

Documentation for specific practical work

[Python Control dlqr](#) - outputs K, S and E, and the discrete objective; [CVXPY: Quadratic program](#) - variables, quad_form, equalities and inequalities.

[OSQP Solver settings](#) - max_iter, time_limit, eps_abs, eps_rel and warm_starting. For task 8, set a time limit, log status and check residuals independently; finding a point does not yet establish successful control.

After reading

For each topic, first write the state vector, matrix dimensions and units, then open the code. Q_cost and Q_noise have different meanings and different names in the project. After reading, implement KF prediction and update and one LQR feedback law, then manually check total force and moment from QP.

Key terms and definitions

State. A vector of variables sufficient to predict the model's subsequent motion given known future inputs.

Operating point. The state and input around which a system is studied; equilibrium additionally requires an unchanging state.

Linearisation. A local approximation of nonlinear equations using first derivatives with respect to state and input around a chosen operating point.

Controllability. The ability to move a linear model between arbitrary states in finite time using an input admissible under the problem formulation.

Observability. The ability to determine a model's initial state uniquely from its output history with known inputs.

Covariance matrix. A matrix describing the joint variability of random errors; the diagonal contains variances, and each element's units are products of component units.

Process noise. A random input representing uncertainty in modelled state evolution; its covariance is denoted Q_{noise} .

Measurement noise. The random component of sensor error in the observation equation; its covariance R_{noise} refers to the measured quantities.

Innovation. The difference between a new measurement and the predicted measurement before updating; it signals disagreement between data and prediction.

Kalman gain. The matrix mapping innovation to a state correction, determined by the measurement model and estimated covariances.

Kalman filter (KF). Recursive state estimation for a linear model through prediction and update; optimality requires stated noise assumptions.

Extended Kalman filter (EKF). An estimator for a nonlinear model using local Jacobians to propagate covariance and update from measurements.

LQR. Feedback that minimises a quadratic cost of state and actuation for a linear model under specified problem conditions.

LQG. The combination of LQR and Kalman estimation in a linear problem with Gaussian noise and a quadratic criterion.

Quadratic programming (QP). Optimisation of a quadratic objective subject to linear equalities and inequalities; convexity requires a positive semidefinite quadratic form.

Feasible solution. A variable vector satisfying every problem constraint within declared numerical tolerances; optimality is checked separately.

Constraint residual. A quantitative measure of equality or inequality violation, calculated from returned variables independently of the solver message.

Fallback control. A predefined admissible action when estimation or optimisation fails, consistent with the system's current limits.

Check the relationships: $x[k+1] = A_d \cdot x[k] + B_d \cdot u[k]$; with $u = -Kx$, the closed-loop matrix is $A_d - B_d K$. Command limiting changes the linear model.

What to install for KF, LQR and QP

NEW PURCHASES: NONE REQUIRED

Use the existing MacBook Pro M4 and models and recordings from M7-M10; the physical rig, IMU and ESP32-S3 have already been purchased. Solve the two-support force-allocation problem numerically. It requires no second physical rig, new joints, GPU or onboard computer.

USE THE PINNED ENVIRONMENT

macOS ARM: Python, NumPy, SciPy, Matplotlib, pytest and control from M10. Retain the selected Python and compatible dependency versions. For repetition on Ubuntu 24.04 ARM64, create a separate environment; do not mix binary packages from different architectures or environments.

INSTALL BEFORE TASKS 7-8

Add **cvxpy** and its available CPU solver **osqp**; verify its presence in `installed_solvers`. Instructions: [CVXPY Install](#). On macOS, use the configured Xcode Command Line Tools. Pin versions after a successful short experiment; there is no need to update the entire environment.

OPTIONAL: BEFORE TASKS 3-4

Add **filterpy** only for independent comparison with your own KF and EKF implementations. [FilterPy: Installation](#) describes release installation; [ExtendedKalmanFilter](#) describes `dim_x` and `dim_z` and the `Hjacobian` and `Hx` functions. Write the core matrix steps yourself first.

CHECK INSTALLATION AND BEHAVIOUR

1. Import packages and record versions and the solver list. 2. Perform one KF prediction-and-update cycle and compare it with a hand calculation. 3. Obtain `dlqr` for the discrete axis and check the poles of `A-BK`. 4. Solve the scalar QP $\min (x-2)^2$ with $0 \leq x \leq 1$: expect $x \approx 1$, then independently check the bounds.

5. Run the module's two-support QP and check forces, moment and friction with declared tolerances. 6. Supply incompatible constraints and a deliberately short computation deadline: the system must enter the specified fallback mode. Check both the solver call and the total command-computation deadline; an individual solver's `time_limit` does not cover the whole loop.

Save `Q_noise`, `R_noise`, `Q_cost` and `R_cost` configuration, `Ts`, constraints and the check record in the README. Installation and preparation are included in the existing 75 hours.

Motion fundamentals: state and support forces

Required: what a sensor allows you to reconstruct

Joint state $x = [q, v]$ stores angle and velocity for prediction. One angle measurement does not give velocity; angle history and a known input allow it to be reconstructed. Before calculating, explain why the same q with different v produces different leg motion, and draw the ground forces on the body through two supports.

From measurement history to velocity

A fixed-base joint without friction or gravitational torque: $q' = v$, $Jv' = u$. q is in rad, v in rad/s, J in $\text{kg}\cdot\text{m}^2$ and u in $\text{N}\cdot\text{m}$ with the same positive direction as q . For constant u over interval T_s , the exact model is $\mathbf{q1} = \mathbf{q0} + \mathbf{T_s}\cdot\mathbf{v0} + \mathbf{T_s^2}u/(2J)$; $\mathbf{v1} = \mathbf{v0} + \mathbf{T_s}\cdot\mathbf{u}/J$. Time, input and parameters must be known.

Let $J = 0.02 \text{ kg}\cdot\text{m}^2$, $T_s = 0.10 \text{ s}$ and $u = 0.04 \text{ N}\cdot\text{m}$; the sensor reports $q_0 = 0.20$ and $q_1 = 0.24 \text{ rad}$. The acceleration contribution to angle is 0.01 rad . Therefore $v_0 = (0.24 - 0.20 - 0.01)/0.10 = \mathbf{0.30 \text{ rad/s}}$, and $v_1 = \mathbf{0.50 \text{ rad/s}}$. A simple angle difference would give the mean velocity 0.40 rad/s , rather than v_0 or v_1 .

When measuring q , the two observability rows are $[1, 0]$ and $[1, T_s]$; they are independent for $T_s \neq 0$. When measuring only v , both rows are $[0, 1]$: an arbitrary constant shift in q is invisible. A Kalman filter cannot create this missing information. With noise, reconstruction becomes estimation with uncertainty, making R , Q and timestamps essential.

From desired motion to admissible forces

Model two point supports in a right-handed frame: x forward, z upward, moment M_y about $+y$. From the centre of mass to the supports, $r_1 = (-a, 0, -h)$, $r_2 = (a, 0, -h)$. Contact forces f act **from the ground on the robot**. If horizontal forces are zero, $F_z = f_{z1} + f_{z2}$ and $M_y = a(f_{z1} - f_{z2})$. These are force and moment sums; QP seeks an admissible allocation.

For $m = 2 \text{ kg}$, $g = 9.81 \text{ m/s}^2$, $a = 0.20 \text{ m}$, $F_z = mg = 19.62 \text{ N}$ and $M_y = 0.40 \text{ N}\cdot\text{m}$, obtain $f_{z1} - f_{z2} = 2 \text{ N}$, hence **$f_{z1} = 10.81 \text{ N}$, $f_{z2} = 8.81 \text{ N}$** . At $M_y = 0$, both forces are 9.81 N . A non-zero resultant moment specifies angular acceleration of the body; this is no longer static equilibrium. With a limit of 12 N per support, a request $F_z = 30 \text{ N}$ is infeasible because the total cannot exceed 24 N .

Support forces are not yet joint commands. Their generalised external action is $J^T f$; the actuator accounts for gravity and required dynamics, and opposes external action when holding. Check only force allocation here.

Exercises and verifiable results

1. In task 2, compare initial $q = 0.20$ and 0.70 rad with the same v and input. Save two angle and velocity histories: v measurements will agree, while q differs by 0.50 rad . **2.** In task 7, compare the manual forces with the solver and record sums and moment; then, in task 8, request 30 N with a hard equality and check infeasible. For the soft problem, check the residual rather than expecting the same status.

Time: 1 theory hour each from 7-8 (observability) and 61-62 (support forces); 1 practice hour each from 15-16 (task 2) and 66-67 (task 7). These calculations belong to existing tasks within the 75 hours; floating-base dynamics comes later.

Read: Lall, EE363, [Observability and state estimation, slides 3-11](#); Lynch, Park, [Modern Robotics §5.2: the relationship between forces and joint torques](#).